

# Use of AI Tools for Part 3

Hadi Daccache Raphael Gastaldo Adrian Rubem Alexandre Johannel

11/23/2025

## 1 Use of AI tools for Part 3 (file `matrix.c`)

For Part 3 of the project which required computing powers of a transition matrix, checking convergence, extracting submatrices corresponding to strongly connected components, and computing the period of each class we used the AI assistant ChatGPT to help refactor and clean the implementation of `matrix.c`.

### 1.1 Scope of the AI assistance

The AI was used only as a support tool to:

- rewrite the code in a clearer and safer style,
- remove the `main()` function from `matrix.c` (since Part 3 is run inside the SDL interface),
- correct memory handling issues, `adj_list.c`),
- remove unnecessary prints and debugging code.

All the mathematical logic (matrix multiplication, difference, period detection, etc.) was preserved exactly as written in my original version.

### 1.2 Prompts used

The main prompt used to generate and correct the file was:

Rewrite the entire `matrix.c` file:

- remove the `main()` function,
- keep only matrix operations (create, free, multiply, diff, print),
- keep the submatrix extraction for Tarjan's classes,
- keep the period computation function,
- ensure clean memory management,

- return a safe and complete version compatible with the rest of the project.

Additional prompts were used to correct edge cases (e.g. power computation, copying matrices, addressing negative indices).

### 1.3 Code obtained

The following is an excerpt of the code produced with AI assistance and integrated into the project. It shows in particular the conversion from adjacency list to matrix and the extraction of a submatrix corresponding to one strongly connected component:

```
t_matrix adjToMatrix(const AdjList *adj) {
    if (!adj) {
        t_matrix empty = (t_matrix){0, NULL};
        return empty;
    }

    int n = adj->n;
    t_matrix mat = matrixCreate(n);

    for (int i = 0; i < n; i++) {
        EdgeCell *curr = adj->L[i].head;
        while (curr) {
            int j = curr->v;          /* 0-based destination */
            float p = curr->p;
            if (j >= 0 && j < n) {
                mat.data[i][j] = p;
            }
            curr = curr->next;
        }
    }
    return mat;
}

/* Extract submatrix for class index */
t_matrix subMatrix(t_matrix matrix, Partition part, int compo_index) {
    if (compo_index < 0 || compo_index >= part.count) {
        fprintf(stderr, "ERROR: _invalid_component_index_%d\n", compo_index);
        t_matrix empty = (t_matrix){0, NULL};
        return empty;
    }

    Class cls = part.classes[compo_index];
    int k = cls.size;
```

```

t_matrix sub = matrixCreate(k);

for (int i = 0; i < k; i++) {
    int oi = cls.vertices[i] - 1; /* convert to 0-based */
    for (int j = 0; j < k; j++) {
        int oj = cls.vertices[j] - 1;
        sub.data[i][j] = matrix.data[oi][oj];
    }
}
return sub;
}

```